academic-pdf-reading v1.3.0 (2026-10-11) — standalone bundle

> Generated from the skill folder by `scripts/build_bundle.py`. **Do not edit by hand**: edit the skill and rebuild. Version string must match the installed skill's `metadata.version`.

When to use: Error-averse protocol for reading academic papers and textbooks stored as PDFs, as ZIP archives of page images plus text layers disguised as .pdf, as text-only extractions, or as pdf-to-markdown transcriptions (NAME.md plus NAME_figures.pdf), whenever equations, indices, signs, overbars, primes or notation must be read exactly. Use this skill whenever the user asks about an equation, derivation, figure, page, section or notation in an uploaded or project-knowledge paper or textbook, asks to quote, check or transcribe a formula from a PDF or a transcription, or when a .pdf file turns out not to be a real PDF. Also use it at the first use in each session when a project-specific protocol (pdf_protocol_*.md) exists, and to create, check or update such a protocol. Covers container triage by magic bytes, text-layer failure modes (character substitution, geometry destruction, OCR), page offsets, image cropping for math, and quoting rules. Ships triage.py for deterministic measurements.

How to use this bundle without the installed skill

1. Extract the script (Part 3) to a writable location:

```
python3 - <<'EOF'
import re
t = open('/mnt/project/academic-pdf-reading.md', encoding='utf-8').read()
s = re.search(r'~~~~python\n(.*?)\n~~~~', t, re.S).group(1)
open('/home/claude/triage.py', 'w', encoding='utf-8').write(s)
EOF
```

(adjust the first path to wherever this bundle lives). 2. Read `SKILL_DIR/scripts/triage.py` as `/home/claude/triage.py`, and `SKILL_DIR/references/project_protocol.md` as Part 2 below.

Part 1 — SKILL.md

Reading academic PDFs without transcription errors

Text extraction from scientific documents fails **silently**. A minus sign vanishes, a superscript `-1` becomes a subtraction, an overbar disappears, a prime turns into the digit `0`, and the result still looks like plausible mathematics. Everything in this skill exists to stop a corrupted text layer from reaching an answer as if it were the paper's content.

A document can also arrive already transcribed: the pdf-to-markdown skill turns a PDF into `NAME.md` (text, with LaTeX math transcribed from the page images) plus `NAME_figures.pdf`. Such a transcription is read as source (§3.6); none of the text-layer machinery below applies to it.

Throughout, `SKILL_DIR` means this skill's directory (for an installed skill, typically `/mnt/skills/user/academic-pdf-reading`). The script is `SKILL_DIR/scripts/triage.py`; run `python3 SKILL_DIR/scripts/triage.py -h` for usage.

---

0. Invariant policy

These hold for every file, every project, every question.

1. **Identify files by magic bytes, never by extension.** A `.pdf` may be a PDF, a ZIP archive, or plain text.
2. **Two channels, two jobs.** The *text channel* is for locating things and for prose. The *image channel* is for mathematics, figures, tables, and anything whose meaning depends on position or typeface.
3. **No equation, sign, index, exponent, coefficient, bar, hat, tilde, prime, delta, dagger, or typeface distinction is ever quoted from a text layer.** It comes off the page image. If a file has no image channel, anything mathematical taken from it is labelled "*text-layer reading, unverified*" and, where possible, cross-checked against another source or re-derived. A pdf-to-markdown transcription is not a text layer: its math was transcribed from the page images and machine-checked when it was made (§3.6).
4. **Source outranks rendering.** If the LaTeX source of a document is available, read the source. A pdf-to-markdown transcription is read the same way: quoted as-is, math included, without re-checking it against page images in the chat (§3.6).
5. **No glyph mapping transfers between documents** — not between two papers of the same group, journal and year, not between parts of one book. Mappings are properties of one file's embedded fonts.
6. **Every reading fact carries its provenance**: "verified at p. N (context)" when checked on the image, "provisional (context only)" otherwise.
7. **Cite the printed page and the equation tag**, and know how the printed page relates to the file's page index (§5). In a transcription the page markers give both (§3.6).
8. **If the image is still ambiguous after cropping, say so.** Cross-check the same object elsewhere (§6) or report the ambiguity; do not guess.
9. **Reading facts belong to a rendition, not to a file name.** The same document can reach you as different containers (§1.1); container-specific facts measured on one never transfer to another.

---

1. Entry: what to do first

Work in a writable copy. Project and upload directories are usually read-only; unpack and crop under `/home/claude` or a temp directory.

If a project protocol exists (a `pdf_protocol_*.md` in project knowledge, or a pointer to one in the user's instructions):

1. Read it.
2. On the **first use of this skill in the session**, run the staleness check:

   ```
   python3 SKILL_DIR/scripts/triage.py check /mnt/project/pdf_protocol_<name>.md
   ```

   Re-run it whenever the user says files were added or replaced, or a file is attached in the chat. Do not re-run it per question.
3. If it reports **stale** (exit 1): tell the user exactly what was added, removed or changed; read the affected files via the no-protocol path below; offer to update the protocol following `references/project_protocol.md`. A stale protocol never blocks answering. If it reports only **probable rendition swaps** (`SWAP?`, exit 3): tell the user which files, and ask whether any of them was edited or replaced. Until they answer, read those files via the no-protocol path — the protocol has no facts for the mounted rendition. If they confirm nothing changed, offer to record the new renditions (`references/project_protocol.md` §6). See §1.1. Only *document* files (PDF, ZIP, text extraction, image) can make a protocol stale. Source-like files are listed separately and do not: source files (`.tex` `.md` `.bib` `.sty` `.cls`, as `source +/-/~`), pdf-to-markdown transcriptions (`transcr. +/~`) and their figures PDFs (`figures +/~`). Read them directly, with no update round. Three cases need a mention to the user:
   - a removed or renamed source that the protocol names (e.g. in a source–rendering pairing);
   - a source or transcription that newly pairs with a document already in the protocol (`pairing` line), since it then outranks that rendering;
   - a recorded document replaced by a transcription of it (`CONVERTED`). This *is* stale, because the protocol still holds reading facts for a rendition that is gone, but it needs no triage: rewrite the entry as `references/project_protocol.md` §6 says. Until then, read the transcription (§3.6) and ignore the old entry's glyph mappings.
4. If it reports **up to date** (exit 0): use the protocol's per-file facts (container, offsets, verified mappings, traps) in place of re-deriving them, taking the facts of the rendition that is mounted (`mounted` lines; the file's magic bytes decide).

If no project protocol exists (a one-off upload, a new project, a file not yet in the inventory): triage each file you need (§2) and proceed with the general strategies below. Do not write a protocol file unless asked. If the same files will keep coming back, suggest generating one.

To create or update a project protocol, read `SKILL_DIR/references/project_protocol.md` and follow it.

1.1 Renditions

A project file can be presented in more than one **rendition** — different containers holding the same document — and which one is mounted can differ between sessions, without the user changing anything. Observed on claude.ai project knowledge: an uploaded PDF mounted as itself in one session and as a paged ZIP of page images plus text layers (§3.1) in another, switching in both directions; after a round trip the file had its original byte size again (the earlier PDF hash had not been kept). The mechanism is not documented; treat it as possible for every project file.

Consequences:

- A changed hash is not by itself a changed document. The check recognises every rendition the inventory records, and classifies an unknown hash under a known name as a **probable swap** only if the container differs from every recorded rendition, the page count matches, and the prose sketch (if one is recorded) agrees. Anything else is **changed**.
- The sketch (per-page simhash of letter 5-grams, `triage.py`) identifies the *document*, not the *version*: it cannot see a revision that edits a few equations or sentences. A probable swap is evidence, not proof. The user decides, which is why exit 3 always ends in a question to the user.
- A new hash in the *same* container as a recorded rendition is reported as changed even if the sketch agrees (it may be a revision touching only math, or a re-render); ask the user.
- Container-dependent facts — access recipe, text-layer glyph and control-byte mappings, line endings, pixel coordinates and column gutters, manifest quirks — are recorded per rendition. Facts about the document itself — printed-page offset, equation-tag ranges, section map — usually hold across renditions but are confirmed once on each new rendition before reuse.

---

2. Triage

```
python3 SKILL_DIR/scripts/triage.py tools                # what exists here
python3 SKILL_DIR/scripts/triage.py profile FILE...      # per-file profile
```

The profile answers three questions. The script **measures**; the answers to questions 2–3 are settled by looking at page images.

Q1 — Container (by magic bytes):

magic	container	meaning
`%PDF-`	genuine PDF	poppler tools work; §3.2
`PK\x03\x04`	ZIP	usually a *paged archive*: `N.jpeg` + `N.txt` per page + `manifest.json`; §3.1. Any other layout: list the members and inspect by hand.
UTF-8 text, no NUL, `.pdf` or other extension	text-only extraction	**no image channel**; §3.3
UTF-8 text whose first line is `<!-- Transcription of NAME.pdf. … -->` (any extension)	transcription	pdf-to-markdown output: read as source; §3.6
UTF-8 text, `.tex` `.md` `.bib` `.sty` `.cls`	source	read directly; §3.4. A `.txt` is **not** assumed to be source: it may be an extraction
`%PDF-`, named `NAME_figures.pdf`, beside a transcription `NAME.md`	figures PDF	figure images of that transcription; §3.6. The profile reports it as `FIGURES PDF of …`
JPEG / PNG	single page image	image channel only
anything else	unknown	**stop and report**; do not guess

Q2 — Channels: image yes/no, text yes/no. This fixes the regime:

image	text	regime
yes	yes	standard: locate in text, read on image

image	text	regime
yes	no	*image-only* (scanned PDF, no text layer): locate by viewing low-resolution pages, then crop. OCR (e.g. `tesseract`) may be used to **locate**, never to quote
no	yes	*unverified*: text is the only source; label all math unverified (§3.3). Not for source files or transcriptions (§3.4, §3.6)
no	no	nothing readable; report

Q3 — Failure modes of the text channel: which of A, B, C (§4) are present. They co-occur; record every one that applies, and apply the union of their restrictions. The profile's `NOTE` lines flag suspects:

- many `ð`/`Þ` and equation tags like `ð40Þ` → substitution (A) suspect;
- `|…i` patterns → substituted closing angle bracket (A) suspect;
- U+21B5 or dropped-ligature words, or Private-Use characters inside prose words → substitution **in prose** (A) suspect;
- a high fraction of 1–2-character lines → geometry destruction (B) suspect;
- control bytes (`\x01`–`\x1f`) → font-slot leakage; map each on the image;
- `GlyphLessFont`, or a scanner/OCR producer string → OCR (C) suspect;
- fonts without ToUnicode maps, or Type 3 fonts → text set in those fonts may extract wrongly.

A suspect is a hypothesis. Confirm it by comparing one displayed equation, one inline formula and one prose paragraph against the image before relying on it.

---

3. Access recipes

3.1 Paged ZIP archive (`N.jpeg` + `N.txt` + `manifest.json`)

- `unzip` (or `triage.py extract FILE --out DIR`) into a writable directory; `pdftotext`, `pdfinfo`, `pdftoppm`, `pdfplumber` do nothing useful on it.
- `manifest.json` lists pages, image paths and dimensions. Treat any "visual content" flag with suspicion until checked; it can be `true` on every page.
- Page `N` is `N.jpeg` / `N.txt`. Check whether the *printed* page equals `N` (§5) — often it does, sometimes with a fixed offset.
- Text layers commonly use CRLF line endings.

3.2 Genuine PDF

- `pdfinfo` (pages, producer), `pdffonts` (embedded fonts, Type 3, ToUnicode), `pdfimages -list` (raster images per page: a full-page image on every page suggests a scan).
- Text: `pdftotext -f N -l N FILE -` (reading order; avoid `-layout` for grepping, it interleaves columns). `triage.py extract` writes one `N.txt` per page for uniform grepping. A form feed `\f` separates pages in whole-document output.
- Image: `pdftoppm -r 200 -f N -l N -png FILE out`, or `triage.py crop`.
- No poppler: text via `pypdf` or `pdfplumber`; rendering via PyMuPDF (`fitz`). If nothing can render, the file is in the *unverified* regime until a renderer is available — say so.
- A LaTeX-produced PDF with proper ToUnicode maps usually has correct *characters* but still flattened geometry (mode B): superscripts on their own lines, big delimiters detached from their contents, evaluation bars dropped, and **drawn figures (TikZ, Feynman diagrams) absent from the text entirely**.

3.3 Text-only extraction (text disguised as `.pdf`)

- Read and grep directly; there is nothing to unzip and no image to consult.
- No page boundaries may exist; `\x0c` is often a glyph, not a page break. Locate pages by running headers (page number + chapter/section title); anchor on number *and* title, since equation lines can start with digits.
- Mixed line endings are common: strip `\r` before anchoring on `$`.
- Every equation taken from such a file is labelled "*text-layer reading, unverified*" and should be (a) cross-checked against the same object in an image-verifiable source, or (b) re-derived. Never settle a sign, bar, prime, dagger, sub/superscript or bra/ket from it alone. When the same formula exists in an image-verifiable file, cite that file for the formula and the text-only file for context.

3.4 Source files

`.tex` and `.md` are the ground truth for their own content and may be quoted as-is, math included. A `.md` that opens with the transcription legend is a pdf-to-markdown transcription; §3.6 says how to read it.

3.5 Viewing and cropping

A full page at native resolution is fine for prose and too small for subscripts. Crop the line in question and upscale 2.5–4× with LANCZOS:

```
python3 SKILL_DIR/scripts/triage.py crop FILE --page 8 --box 55 150 480 330 --scale 3 --out /home/claude/c.p
ng
```

then open the PNG with the `view` tool. `--box` is in pixels of the page image (for a PDF, at `--dpi`, default 200); the command prints the page size. On a two-column page, find the column gutter once and reuse it. For every sign, index, prime, bar, fraction, or permutation subscript, crop *that line*; resolve superscript vs subscript and numerator vs denominator explicitly.

3.6 Markdown transcriptions (pdf-to-markdown)

Recognition. The first line is the legend `<!-- Transcription of NAME.pdf. "page N" markers give the PDF page … -->`. `triage.py profile` reports the container as `transcription` and counts its page markers, `\tag`s and figure lines. The transcribed PDF can carry another name in project knowledge (uploads rename files); the check matches them by letters and digits only.

Trust. The transcription is read as source: quote it as-is, math included, and label it by page (`transcription, p. N`), not as `verified` or `unverified`. Do not re-check its equations against page images in the chat; the point of converting is to spend those tokens once, at conversion time, where every formula was transcribed from the image and proofread. Its LaTeX keeps typeface distinctions (`\mathcal{G}` vs `G`, `\boldsymbol`), bars, primes and tags as printed, apparent typos included.

When to escalate. Only in the following cases. Say what the transcription says and ask the user for the original page. Do not correct it silently or guess.

- A derivation, or a companion paper that restates the same object, contradicts it, and the conflict survives a re-check of the derivation.
- It is internally inconsistent: a tag missing or duplicated, a formula that does not parse, a sentence that breaks off.
- The profile reports a structural defect: missing or out-of-order page markers, a stray `<!-- continues -->`, a figures PDF that is not there.

Pages. `<!-- page N -->` opens page N of the PDF. `<!-- page N (printed M) -->` adds the number printed on it. Where a paragraph runs across a break, the marker stands inline at the break, so the text after it is on page N. Without `(printed M)`, the printed number equals N or was not determined at conversion; a project protocol records which. Otherwise cite N and say it is the PDF page. `triage.py grep PATTERN FILE` reports hits as `pN` or `pN(printed M)`, and `triage.py text FILE N` prints page N.

Equations. Displays carry `\tag{N}` with the printed equation number, so grep `\\tag\{12a\}`. In-text references keep the document's own style.

Figures. A figure is a placeholder line `*[Fig. N: NAME_figures.pdf, page P]*` followed by its transcribed caption. Text inside the figure (axis labels, legends, diagram labels) is not in the Markdown. For any question about what a figure shows, open page P of the figures PDF. Each figure there is labelled `Fig. N, p. S` or `Fig. N, p. S (printed M)` (S its PDF page), and several figures share a page. Crop one with `triage.py crop` if labels are small. The figures PDF has nothing else to triage.

Beside the original. If the transcribed PDF is also mounted, read the transcription for content. Open the PDF only for questions about the page itself (layout, an element the transcription omits).

---

4. Failure modes of the text channel

Mode A — character substitution

Mechanism. A font is re-encoded, so glyphs extract as *other, legible* characters. The damage looks like text, which is what makes it dangerous.

Signature. Unusual Latin-1 letters where delimiters belong (`ð…Þ` for parentheses, `¼` for `=`), equation tags in the substituted form, kets closing with `i`, ligatures turned into arrows, Private-Use characters or nothing.

What is safe. Prose set in an unaffected text font. Check: if prose parentheses are ASCII while math parentheses are `ð…Þ`, only the math font is remapped. If prose words are damaged (ligatures, `A\ue00biliations`), prose is *not* automatically safe; repair it only when the completion is unique.

Grep strategy. Search for the *substituted* form of math tokens. An equation's display tag uses the math font (`ð40Þ`), while in-prose cross-references use the text font (`Eq. (40)`): grepping `(40)` finds citations, not the definition.

Traps.

- Ordinary letters and digits can be remapped delimiters or symbols: `f g` for `{ }`, `h i j` for `⟨ ⟩ |`, `0` for a prime, `l` for `ℓ`. These are *two-valued* against real words and numbers.
- One control byte can stand for several unrelated glyphs in the same file.

Illustration (Kugler et al., PRX 11, 041006 (2021), p. 15; mechanism only, not a mapping for any other file): `ci ∈ f\x04g` is $c_i \in \{\pm\}$; `Fσσ0` is $F_{\sigma\sigma'}$; `2l` is 2^{ℓ}; and `\x02` is a line-break hyphen in `compo\x02nents`, a cdot in `ω1\x02\x02\x02λ`, and a big parenthesis around the matrix of Eq. (75).

Mode B — geometry destruction

Mechanism. Characters come out right; their *positions* do not.

Signature. Many 1–2-character lines; displayed equations split across lines; matrices without rows.

What is safe. Prose; the identity of individual symbols (after checking a sample); section headers; equation tags.

Traps.

- **False binary operations**: a flattened superscript reads as arithmetic. `p–1(η)` is $p^{-1}(\eta)$; `β–1` is β^{-1}; `g 0` is g^{0}. This is the most common misreading.
- **Accents, bars, hats, tildes dropped or detached**, and not uniformly within one expression: one bar survives as a free-standing `¯` before or after its index, the next vanishes.
- **Typeface distinctions erased**: calligraphic $\mathscr{G}$ and roman G, bold $\boldsymbol{\omega}$ and ω, $\mathscr{T}$ and T can extract identically. Where the notation distinguishes them, the text layer silently merges two different objects.
- **Big delimiters detached** from their contents and **evaluation bars dropped** (`G(1,·) U 2` for G^{(1,·)}|_{U²}).
- **Control bytes are font-slot leakage** with per-document meaning (often big delimiters, Greek capitals, integral and sum signs, primes, relations). One byte can be several glyphs in one file: a hyphen, Σ and ∫, or ⟩, Θ and ∫.
- **Mathematical-alphanumeric Unicode** (italic ω, bold $\boldsymbol{\omega}$) is searchable only literally; Private-Use characters have no stable meaning.
- **Column order**: two-column pages usually extract left column fully, then right — but verify once per file; footers and running heads can splice into the middle of a hyphenated word.
- **Lost inter-word spaces** in tightly justified lines: search for one distinctive word, not a multi-word phrase.

Mode C — OCR (probabilistic)

Mechanism. The text layer was produced by recognising a scan.

Signature. `GlyphLessFont`, scanner/OCR producer strings, a full-page raster on every page, plausible-but-wrong words, digits confused with letters.

Strategy. Prose is no longer automatically safe and has no deterministic repair: verify any prose sentence you quote on the image. Use fuzzy search (distinctive words, partial matches) to locate. Mathematics is image-only, as always.

Co-occurrence. A file can show A and B together (substituted kets inside flattened equations), or A in prose and B in math. Record all modes present.

---

5. Page numbering

Printed page numbers and file page indices differ more often than expected.

1. Run `triage.py profile`; read the `folio` lines. Each pattern (e-locator `041006-N`, `(N of T)`, `- N -`, footer or running-head numbers) reports the dominant offset, how many pages agree, and which disagree or lack a match.
2. If one pattern agrees on all pages, the offset is verified exhaustively. Otherwise confirm the offset on the image at no fewer than three spread pages, and note unnumbered pages (openers, figure pages).
3. Watch for segments: front matter, appendices or an index can carry a different offset or none.

4. Text-only files: use running headers (`running_headers` in the profile); they are noisy, so confirm a located page by its section title.
5. Transcriptions: the page markers give the PDF page and, where it differs, the printed page (§3.6); nothing to measure.

---

6. Workflow: locate → read → quote

Locate (text channel).

- Grep for a distinctive *prose* word or phrase; section headers are usually clean.
- For an equation's definition, grep its tag in the form the file uses (ASCII `(146a)`, substituted `ð40Þ`, bracketed `[1]`, or `\tag{1}` in a transcription). Know the citation style too: some journals tag equations `[N]` and cite references `(N)`.
- `triage.py grep PATTERN FILE... [--escape]` reports `file:pPAGE:LINE`, strips `\r`, and with `--escape` shows control bytes as `\xNN`.
- `triage.py text FILE N --escape` dumps one page with damage visible.

Read (image channel). Open the page; crop every line that carries a sign, index or accent (§3.5). When still in doubt, cross-check the same object where it reappears — later in the same document, or in a companion paper that restates it.

Quote.

- Prose may be quoted from the text layer after deterministic repairs (line-break hyphen bytes → `-` or joined; ligature damage with a unique completion), except under mode C.
- Mathematics is quoted from the image, from source, or from a transcription (§3.6); anything else is labelled unverified.
- Cite as "Author Year, p. N, Eq. (X)" with N the printed page.

---

7. Establishing a glyph mapping

1. Find the glyph in the text layer (`grep --escape`).
2. Locate the same spot on the image and crop it.
3. Record `X = Y, verified at p. N (context)`.
4. Test multi-valuedness: sample at least three contexts of different kinds (prose, inline math, display math, delimiters). If they disagree, record every meaning with its context.
5. Anything inferred from context alone is `provisional`. For a text-only file that is the best available; say so.
6. Never apply a mapping from one file to another.

---

8. Tool fallbacks

need	first choice	fallback	none available
container	`triage.py profile`	`head -c 8 FILE | od -c`	—
PDF text	`pdftotext`	`pypdf`, `pdfplumber`	image-only regime
PDF image	`pdftoppm`	PyMuPDF	unverified regime; say so
crop	`triage.py crop` (Pillow)	view whole page	view whole page, flag low confidence

need	first choice	fallback	none available
fonts	`pdffonts`	`pypdf` font resources	skip; rely on image checks
OCR (locate only)	`tesseract`	—	view pages sequentially

Part 2 — references/project_protocol.md

Generating and maintaining a project-specific PDF protocol

Companion to `SKILL.md` (academic-pdf-reading v1.3.0). Read this when asked to **create**, **update**, or **review** a project protocol, or when the staleness check reports a mismatch and the user wants the protocol brought up to date.

Contents

1. What a project protocol is for
2. What belongs in it, and what does not
3. Creating one
4. Verification standards
5. Hand-off and read-back
6. Updating one
7. Template

---

1. What a project protocol is for

The skill holds mechanisms; the project protocol holds **measured facts about this project's files**: which container each one is, which failure modes its text layer shows, its page offset, which glyph mappings have been verified on the image, and which notation in this project is most at risk. It caches work that would otherwise be repeated every session, and it carries a machine- checkable inventory so that a changed file set is detected before stale facts are used.

2. What belongs in it, and what does not

Belongs: the inventory (per file, one or more renditions, each with SHA-256, size, container, pages and prose sketch; see `SKILL.md` §1.1); per-file profile, split by rendition where it depends on the container; verified and provisional mappings with provenance; page offsets and their basis; structural maps (where appendices, references, index start); per-file grep conventions (equation-tag form, citation form); project-specific traps; a changelog.

For a **pdf-to-markdown transcription** (`SKILL.md` §3.6) the entry is short, because there is nothing to triage:

- the document it transcribes, and the PDF it replaced in this project, if any;
- pages covered, and whether printed numbers differ from PDF pages (`printed` in the markers, or "printed = PDF page, verified at conversion");
- its figures PDF;
- any structural defect the profile reports;
- where the original lives, if the user keeps it outside the project. Its figures PDF needs no entry of its own beyond the inventory line.

Does not belong:

- *General mechanisms* already in `SKILL.md` — refer to them ("mode B applies") instead of restating them. Restated rules drift.
- *Content knowledge* — errata in the papers, physics insights, derivations. Those go in the project's notes or chat summaries. The protocol is about *reading*, not about what the documents *say*.
- *Unverified claims presented as facts.* Every mapping is either `verified at p. N` or `provisional`.

3. Creating one

Work through the files in order; keep notes as you go.

1. **Inventory and profile.**

   ```
   python3 SKILL_DIR/scripts/triage.py profile /mnt/project/* --json /home/claude/profile.json
   python3 SKILL_DIR/scripts/triage.py inventory /mnt/project/* --root /mnt/project --out /home/claude/in
   v.json
   ```

   Exclude the protocol file itself if it already exists (the checker ignores `pdf_protocol_*.md` automatically).
2. **Container and channels** for each file, from the profile. Note anything surprising (a `.pdf` that is text; a ZIP that is not a paged archive). Transcriptions and their figures PDFs skip steps 3–6: record their entry as §2 says, from the profile's `transcr.` line and notes.
3. **Failure modes.** For each file with a text channel, compare against the image: one displayed equation, one inline formula, one prose paragraph, and one figure caption. Record which of modes A, B, C are present, and where prose is or is not safe.
4. **Page offset.** From the profile's `folio` lines; confirm per §5 of `SKILL.md`. Record the basis ("e-locator folio, all 32 pages agree").
5. **Glyph mappings.** For every control byte, Private-Use character and substitution candidate the profile lists *and that matters for reading* (appears in math or in prose you will quote), establish the mapping per §7 of `SKILL.md`. Test multi-valuedness. Record provenance. Rare glyphs may be left "unmapped" explicitly.
6. **Grep conventions.** Equation-tag form (ASCII, substituted, bracketed), citation form, column order, line endings.
7. **Structural map.** Section headers; where appendices, references and any index begin.
8. **Project-specific traps.** Ask: which notational features are load-bearing in *these* documents (overbars on indices, primes, calligraphic vs roman letters, Keldysh or contour labels, bold vectors), and does each file's text layer preserve them? Give one verified example per trap.
9. **Write the file** from the template (§7), named `pdf_protocol_<project>.md`, with the inventory JSON block embedded verbatim.
10. **Self-check** before handing it over:

    ```
    python3 SKILL_DIR/scripts/triage.py check /home/claude/pdf_protocol_<project>.md --root /mnt/project
    ```

    must report *up to date*.

4. Verification standards

- **Exhaustive where cheap.** Page offsets from a folio pattern: check every page (the script does). File hashes: every file.
- **Image-verified where it matters.** Every mapping used for math: at least one image check; at least three contexts of different kinds before calling a glyph single-valued.

- **Honest labels.** `verified at p. N (context)`, `provisional (context only)`, `unmapped`. Text-only files can only ever be `provisional` for math.
- **Record negative results** that save future effort ("manifest `has_visual_content` is true on all pages: useless"; "pdftoppm does not apply").

5. Hand-off and read-back

The protocol cannot be saved into project knowledge by Claude (project files are read-only, and skills are global). So:

1. Write the file under `/mnt/user-data/outputs/` and present it.
2. The user uploads it to project knowledge, and adds to custom instructions a pointer such as:

   > PDF reading: follow the academic-pdf-reading skill. Project-specific facts are in `pdf_protocol_<project>.md` in project knowledge; run its staleness check on first use each session.

3. In the next session, the first use of the skill confirms the read-back: the file is present, its JSON block parses, and `triage.py check` passes. If the upload altered the file (run `triage.py profile` on it), say so.

6. Updating one

Triggered by a stale check, or by the user.

Source-like changes (`source`, `transcr.` and `figures` lines in the check output) never trigger an update on their own; the inventory may lag behind them. Fold them in at the next update that happens for another reason: add inventory entries for new sources, record any source–rendering pairing in the paired document's entry ("`X.tex` supersedes this file for content; keep the rendering for figures"), and drop entries for removed sources and any pairing that names them.

- **Added file:** full creation procedure (§3, steps 2–8) for that file; add its inventory entry.
- **Removed file:** delete its entry and every fact that depends on it; note the removal in the changelog (future readers should know it once existed).
- **Converted file** (`CONVERTED X -> Y.md`: the document X is gone and a transcription Y of it is present): no triage.
    1. Retire all of X's renditions and every container-dependent fact (glyph mappings, control bytes, pixel boxes, line endings).
    2. Replace X's entry by a transcription entry for Y (§2).
    3. Keep from X's document-level facts only what still helps locating in Y, e.g. that in-text equation references use `[N]`.
    4. Remove X from traps that cite it as an example, unless another file still illustrates the trap.
    5. Changelog line: "X replaced by its transcription Y".
- **Changed file** (same name, different hash, reported `CHANGED`): treat as new. **Do not assume earlier mappings still hold**; the container itself may have changed. Regenerate its entry without `--add-rendition`, which retires its old renditions and every fact recorded for them.
- **Probable rendition swap** (`SWAP?`): first ask the user whether the file was edited or replaced. If it was, handle it as a changed file. If not:
    1. triage the mounted rendition (§3, steps 2–6) and record its container-dependent facts in a new rendition block of that file's entry (template, §7), leaving the existing rendition's block

untouched;
2. confirm the document-level facts (page offset, tag range, structure) on at least three spread pages of the new rendition and note that they hold for both;
3. append it to the inventory with `triage.py inventory ... --merge OLD --add-rendition NAME...`, naming only the files the user confirmed. A rendition block whose facts were never measured is labelled "facts not yet recorded" rather than omitted, so a later session does not borrow another rendition's mappings.

- Regenerate the inventory block with `triage.py inventory`, using `--merge` with the current protocol so that recorded renditions are kept, bump the protocol's date, add a changelog line, re-run the self-check, hand off (§5).

7. Template

```
# PDF reading protocol — <project>

Generated <date> with academic-pdf-reading v<version> (triage.py <version>).
General rules live in the skill; this file records facts about this project's
files only. Provenance labels: **verified at p. N** (checked on the page
image), **provisional** (context only), **unmapped**.

## Session start
Run on first use each session (and when files change):
`python3 SKILL_DIR/scripts/triage.py check /mnt/project/pdf_protocol_<project>.md`

## Inventory summary
| file (short) | container | channels | pages | printed-page offset (basis) | modes |
|---|---|---|---|---|---|

## Per-file facts
### <short name>
Document-level (hold for every rendition; confirmed on each):
- Printed-page offset (basis):
- Grep conventions: equation-tag range and form; citations.
- Structure:

#### Rendition: <container> (sha256 <first 12 hex>)
- Access:
- Text layer: modes present; what is safe; line endings; column order.
- Verified mappings:
- Provisional / unmapped:
- Geometry: image size or dpi; column gutter.

(Repeat the rendition block for each recorded rendition.)

### <short name> — transcription
- `<NAME>.md`, pdf-to-markdown transcription of <document>; replaces
  `<original file>` (removed <date>) / sits beside it.
- Pages 1–N; printed numbers: <from markers | equal PDF pages, verified at
  conversion>. Equation tags `\tag{…}`; in-text references `<style>`.
- Figures: `<NAME>_figures.pdf`, P pages.
- Defects: <none | from the profile>. Original kept at: <location, if any>.

## Project-specific traps
1. <trap>: <why it matters here>; example verified at <file> p. N.

## Changelog
- <date>: created.

## Inventory (machine-readable; used by `triage.py check`)
```json
{ "pdf_protocol_inventory": { ... } }
```
```

Part 3 — scripts/triage.py

```
#!/usr/bin/env python3
"""
triage.py -- deterministic measurements for reading academic PDF-like files.

This script MEASURES. It never decides what a glyph means or which failure
mode a file has; it reports signals that must be confirmed on the page image.

Subcommands
  tools                           which helper tools exist in this environment
  profile FILE... [--json OUT]    container, channels, text-layer signals,
                                  page-offset candidates, notes
  inventory FILE... [--root DIR] [--out OUT] [--merge OLD]
                    [--add-rendition NAME...]
                                  compact inventory for a project protocol:
                                  per file, one or more renditions (sha256,
                                  size, container, pages, prose sketch).
                                  --merge keeps OLD's renditions for files whose
                                  hash is already known; a file named in
                                  --add-rendition gets its new hash appended as
                                  a further rendition instead of replacing the
                                  old ones (only after the user has confirmed
                                  the content is unchanged).
  check INVENTORY [--root DIR] [--json]
                                  staleness check. INVENTORY is a .json file
                                  or a .md containing a ```json block with the
                                  key "pdf_protocol_inventory". A file matching
                                  any recorded rendition is unchanged. Exit 1 if
                                  stale; exit 3 if the only differences are
                                  probable rendition swaps (confirm them).
                                  Source files (.tex .md ...), pdf-to-markdown
                                  transcriptions and their figures PDFs are
                                  reported but never make the protocol stale;
                                  a recorded document replaced by a
                                  transcription of it is reported CONVERTED
                                  (stale: its entry must be rewritten).
  text FILE [N] [--lines A:B] [--escape]
                                  print the text layer of page N (paged files;
                                  for a transcription, the text between its
                                  page markers N and N+1) or a line range
                                  (text-only files)
  grep PATTERN FILE... [-i] [--escape]
                                  regex search of text layers, reported as
                                  file:page:line (a transcription reports the
                                  page from its markers, e.g. p3 or
                                  p3(printed 1203)); CRLF-safe
  extract FILE --out DIR          paged ZIP -> its members; genuine PDF ->
                                  one N.txt per page via pdftotext
  crop SRC --page N [--box x0 y0 x1 y1] [--scale S] [--dpi D] --out PNG
                                  page image (paged ZIP, genuine PDF, or image
                                  file), optionally cropped and LANCZOS-upscaled

Exit codes: 0 ok; 1 check found differences; 2 usage or I/O error;
3 check found only probable rendition swaps.
Python 3.8+. Optional: poppler-utils (pdfinfo pdftotext pdftoppm pdffonts
pdfimages), Pillow, pypdf.
"""

import argparse
import collections
import datetime
import hashlib
import io
import json
```

```
import os
import re
import shutil
import subprocess
import sys
import tempfile
import unicodedata
import zipfile

TRIAGE_VERSION = "1.3.0"

# ---------------------------------------------------------------- utilities

CLI_TOOLS = ["pdfinfo", "pdftotext", "pdftoppm", "pdffonts", "pdfimages",
             "unzip", "tesseract"]
PY_MODULES = ["PIL", "pypdf", "pdfplumber", "fitz"]


def have(tool):
    return shutil.which(tool) is not None


def has_module(name):
    try:
        __import__(name)
        return True
    except Exception:  # noqa: BLE001
        return False


def tool_report():
    return {"cli": {t: have(t) for t in CLI_TOOLS},
            "python": {m: has_module(m) for m in PY_MODULES}}


def run(cmd, timeout=300):
    try:
        p = subprocess.run(cmd, capture_output=True, timeout=timeout)
        return (p.returncode, p.stdout.decode("utf-8", "replace"),
                p.stderr.decode("utf-8", "replace"))
    except Exception as e:  # noqa: BLE001
        return 99, "", str(e)


def sha256_of(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def escape(s):
    """Make invisible damage visible: control bytes, PUA, and CR."""
    out = []
    for ch in s:
        o = ord(ch)
        if ch == "\r":
            out.append("\\r")
        elif o < 32 and ch not in "\n\t":
            out.append("\\x%02x" % o)
        elif 0xE000 <= o <= 0xF8FF:
            out.append("\\u%04x" % o)
        else:
            out.append(ch)
    return "".join(out)

# ---------------------------------------------------------------- container
```

```
# A pdf-to-markdown transcription is recognised by content, not by extension:
# its first non-blank line is the legend written by assemble.py,
#   <!-- Transcription of NAME.pdf. "page N" markers give the PDF page ... -->
# Page markers  <!-- page N -->  or  <!-- page N (printed M) -->  open every page
# (inline where a paragraph runs across the break); figures are referenced by
# lines  *[Fig. N: NAME_figures.pdf, page P]*  .
TRANSCRIPTION_RX = re.compile(r"\A\s*<!--\s*Transcription of (.+?\.pdf)\.", re.I)
PAGE_MARK_RX = re.compile(r"<!--\s*page\s+(\d+)(?:\s*\(printed\s+([^)]*?)\s*\))?\s*-->")
FIG_LINE_RX = re.compile(r"^\*\[Fig\.\s*([^:\]]+?):\s*(.+?\.pdf),\s*page\s+(\d+)\]\*\s*$", re.M)
CONTINUES_MARK = "<!-- continues -->"
FIGURES_SUFFIX = "_figures.pdf"
SOURCE_LIKE = ("source-text", "transcription")


def norm_stem(name):
    """File name reduced for matching across upload renamings: NFKC, lowercase,
    extension dropped, letters and digits only ('[Lihm 2026] The ...pdf' and
    'Lihm_2026_The_...pdf' agree)."""
    stem = re.sub(r"\.(pdf|md)$", "", name, flags=re.I)
    stem = unicodedata.normalize("NFKC", stem).lower()
    return "".join(ch for ch in stem if ch.isalnum())


def transcription_info(path):
    """Structure of a pdf-to-markdown transcription (measurements only)."""
    t = get_whole_text(path).replace("\r", "")
    m = TRANSCRIPTION_RX.match(t)
    marks = [(int(a), b or None) for a, b in PAGE_MARK_RX.findall(t)]
    nums = [n for n, _ in marks]
    figs = FIG_LINE_RX.findall(t)
    cont_lines = [i for i, ln in enumerate(t.split("\n"), 1) if CONTINUES_MARK in ln]
    tags = re.findall(r"\\tag\{([^}]*)\}", t)
    dup_tags = sorted({x for x in tags if tags.count(x) > 1})
    here = os.path.dirname(os.path.abspath(path))
    fig_files = sorted({f for _, f, _ in figs})
    return {
        "source_pdf": m.group(1) if m else None,
        "page_markers": len(marks),
        "pages": (min(nums), max(nums)) if nums else None,
        "pages_missing": sorted(set(range(min(nums), max(nums) + 1)) - set(nums)) if nums else [],
        "pages_out_of_order": nums != sorted(nums),
        "printed_given": sum(1 for _, p in marks if p),
        "figure_lines": len(figs),
        "figure_files": fig_files,
        "figure_files_missing": [f for f in fig_files if not os.path.isfile(os.path.join(here, f))],
        "tags": len(tags),
        "duplicate_tags": dup_tags[:20],
        "stray_continues_lines": cont_lines,
    }


def transcription_pages(text):
    """[(N, printed, text)] for the segments between page markers. Text before
    the first marker (the legend) is dropped."""
    out, it = [], list(PAGE_MARK_RX.finditer(text))
    for i, m in enumerate(it):
        end = it[i + 1].start() if i + 1 < len(it) else len(text)
        out.append((int(m.group(1)), m.group(2) or None, text[m.end():end]))
    return out


def figures_owner(path):
    """For NAME_figures.pdf lying next to a transcription NAME.md: that .md's
    name; else None. Pairing is by file name, as pdf-to-markdown writes it."""
    base = os.path.basename(path)
    if not base.lower().endswith(FIGURES_SUFFIX):
        return None
    md = os.path.join(os.path.dirname(os.path.abspath(path)), base[:-len(FIGURES_SUFFIX)] + ".md")
```

```
    if os.path.isfile(md) and detect_container(md)[0] == "transcription":
        return os.path.basename(md)
    return None


def file_kind(path):
    """Container, refined by role: 'figures' for a transcription's figures PDF."""
    cont = detect_container(path)[0]
    if cont == "pdf" and figures_owner(path):
        return "figures"
    return cont


def detect_container(path):
    """(container, detail), decided by magic bytes -- never by extension."""
    with open(path, "rb") as f:
        head = f.read(8192)
    if head.startswith(b"%PDF-"):
        return "pdf", head[:8].decode("latin-1").strip()
    if head.startswith(b"PK\x03\x04") or head.startswith(b"PK\x05\x06"):
        return "zip", "PK"
    if head.startswith(b"\xff\xd8\xff"):
        return "image", "jpeg"
    if head.startswith(b"\x89PNG"):
        return "image", "png"
    if b"\x00" not in head:
        try:
            head.decode("utf-8")
            ok = True
        except UnicodeDecodeError as e:
            ok = e.start > len(head) - 4   # multibyte char cut at buffer end
        if ok:
            m = TRANSCRIPTION_RX.match(head.decode("utf-8", "ignore"))
            if m:
                return "transcription", m.group(1)
            ext = os.path.splitext(path)[1].lower()
            if ext in (".tex", ".md", ".bib", ".sty", ".cls"):  # .txt is NOT source: may be an extraction
                return "source-text", ext
            return "text-extraction", ext or "(no extension)"
    return "unknown", repr(head[:8])


def zip_layout(zf):
    names = zf.namelist()
    img = sorted(int(m.group(1)) for n in names
                 for m in [re.fullmatch(r"(\d+)\.(?:jpe?g|png)", n)] if m)
    txt = sorted(int(m.group(1)) for n in names
                 for m in [re.fullmatch(r"(\d+)\.txt", n)] if m)
    contiguous = bool(img) and img == list(range(1, len(img) + 1))
    return {"members": len(names), "page_images": len(img), "page_texts": len(txt),
            "manifest": "manifest.json" in names,
            "paged_archive": contiguous and (not txt or txt == img),
            "other_members": [n for n in names
                              if not re.fullmatch(r"\d+\.(?:jpe?g|png|txt)", n)
                              and n != "manifest.json"][:20]}

# ------------------------------------------------------------------ text access


def zip_pages(path):
    with zipfile.ZipFile(path) as zf:
        d = {}
        for n in zf.namelist():
            m = re.fullmatch(r"(\d+)\.txt", n)
            if m:
                d[int(m.group(1))] = zf.read(n).decode("utf-8", "replace")
    return [d[k] for k in sorted(d)]
```

```
def pdf_npages(path):
    if have("pdfinfo"):
        rc, out, _ = run(["pdfinfo", path])
        m = re.search(r"Pages:\s+(\d+)", out)
        if m:
            return int(m.group(1))
    if has_module("pypdf"):
        import pypdf  # noqa: WPS433
        return len(pypdf.PdfReader(path).pages)
    return 0


def pdf_pages(path, npages=None):
    npages = npages or pdf_npages(path)
    if have("pdftotext"):
        rc, out, _ = run(["pdftotext", "-enc", "UTF-8", path, "-"])
        if rc == 0:
            parts = out.split("\f")
            if len(parts) >= npages:
                return parts[:npages]
    if has_module("pypdf"):
        import pypdf  # noqa: WPS433
        return [(p.extract_text() or "") for p in pypdf.PdfReader(path).pages]
    return None


def get_pages(path):
    """List of per-page texts, or None if the file has no page structure."""
    cont, _ = detect_container(path)
    if cont == "zip":
        with zipfile.ZipFile(path) as zf:
            if not zip_layout(zf)["paged_archive"]:
                return None
        return zip_pages(path)
    if cont == "pdf":
        return pdf_pages(path)
    return None


def get_whole_text(path):
    with open(path, "rb") as f:
        return f.read().decode("utf-8", "replace")

# ------------------------------------------------------------------ signals

DROPPED_LIGATURE_WORDS = ["coecient", "eect", "dierent", "diculty", "sucient",
                          "ecient", "signicant", "denition", "specic",
                          "uctuation", "Pfaan"]


LIGATURE_RX = re.compile(r"\b(?:%s)s?\b" % "|".join(DROPPED_LIGATURE_WORDS))


def text_signals(text):
    c = collections.Counter(text)
    lines = text.split("\n")
    nonempty = [ln.strip("\r") for ln in lines if ln.strip()]
    short = sum(1 for ln in nonempty if len(ln.replace(" ", "")) <= 2)
    crlf = text.count("\r\n")
    between = collections.Counter(m.group(1) for m in re.finditer(
        r"(?<=[a-z])([\x01-\x08\x0b\x0e-\x1f])(?=[a-z])", text))
    eth_tags = len(re.findall("\u00f0\\d+[a-z]?\u00de", text))
    return {
        "chars": len(text), "lines": len(lines),
        "line_endings": {"CRLF": crlf, "LF_only": text.count("\n") - crlf},
        "form_feeds": c.get("\x0c", 0),
        "control_bytes": {"\\x%02x" % ord(k): v for k, v in sorted(c.items())
                          if ord(k) < 32 and k not in "\n\r\t"},
        "control_bytes_between_letters": {"\\x%02x" % ord(k): v for k, v in between.items()},
```

```
        "private_use": sum(v for k, v in c.items() if 0xE000 <= ord(k) <= 0xF8FF),
        "math_alphanumeric": sum(v for k, v in c.items() if 0x1D400 <= ord(k) <= 0x1D7FF),
        "combining_marks": sum(v for k, v in c.items() if 0x0300 <= ord(k) <= 0x036F),
        "greek": sum(v for k, v in c.items() if 0x0370 <= ord(k) <= 0x03FF),
        "unicode_minus": c.get("\u2212", 0),
        "free_standing_macron": c.get("\u00af", 0),
        "substitution_candidates": {
            "eth": c.get("\u00f0", 0), "THORN": c.get("\u00de", 0),
            "thorn": c.get("\u00fe", 0), "one_quarter": c.get("\u00bc", 0),
            "one_half": c.get("\u00bd", 0), "eqtags_eth_thorn": eth_tags,
            "ket_closed_by_i": len(re.findall(r"\|[^|\s]{1,12}i\b", text)),
            "return_arrow": c.get("\u21b5", 0),
            "dropped_ligature_words": len(LIGATURE_RX.findall(text)),
        },
        "short_line_fraction": round(short / max(1, len(nonempty)), 3),
        "eqtag_styles": {
            "ascii_own_line": len(re.findall(r"(?m)^\(\d+[a-z]?\)\r?$", text)),
            "ascii_line_end": len(re.findall(r"(?m)\(\d+[a-z]?\)\r?$", text)),
            "appendix_style": len(re.findall(r"\([A-Z]\.?\d+[a-z]?\)", text)),
            "eth_thorn": eth_tags,
            "bracket_line_end": len(re.findall(r"(?m)\[\d+[a-z]?\]\r?$", text)),
        },
    }


def notes_for(rec):
    """Neutral notes. Signals, never verdicts."""
    n = []
    cont, ch, s = rec["container"], rec["channels"], rec.get("text_signals")
    if cont == "unknown":
        n.append("UNRECOGNISED CONTAINER: stop and report; do not guess.")
    if cont == "zip" and not rec.get("zip_layout", {}).get("paged_archive"):
        n.append("ZIP without the N.jpeg/N.txt layout: list and inspect members by hand.")
    if cont == "source-text":
        n.append("SOURCE FILE: read directly; it outranks any rendering of it.")
    if cont == "transcription":
        ti = rec.get("transcription", {})
        n.append("TRANSCRIPTION of %s (pdf-to-markdown): read as source (SKILL.md 3.6); "
                 "quote as-is, math included; cite pages from its markers."
                 % ti.get("source_pdf"))
        if not ti.get("page_markers"):
            n.append("No page markers: printed pages cannot be cited; cite section and equation tag.")
        if ti.get("pages_missing") or ti.get("pages_out_of_order"):
            n.append("Page markers incomplete or out of order (missing %s): check the assembly."
                     % ti.get("pages_missing", [])[:10])
        if ti.get("stray_continues_lines"):
            n.append("Stray '<!-- continues -->' on line(s) %s: a page join failed; the prose "
                     "there may be cut (a structural defect, not a math one)."
                     % ti["stray_continues_lines"][:10])
        if ti.get("duplicate_tags"):
            n.append("Equation tags occurring more than once: %s." % ti["duplicate_tags"])
        if ti.get("figure_files_missing"):
            n.append("Figures PDF %s not next to this file: figures cannot be viewed."
                     % ti["figure_files_missing"])
    if rec.get("figures_of"):
        n.append("FIGURES PDF of %s: figure images only (its text is just the labels); "
                 "nothing to triage. Open the page its placeholder names." % rec["figures_of"])
    if cont == "text-extraction":
        n.append("TEXT-ONLY EXTRACTION (despite its extension): no image channel exists.")
    if not ch["image"] and cont not in SOURCE_LIKE:
        n.append("NO IMAGE CHANNEL: mathematics is text-layer only -> label unverified.")
    if cont == "pdf" and not rec.get("figures_of"):
        f = rec.get("fonts", {})
        if f.get("no_tounicode"):
            n.append("Fonts without ToUnicode maps %s: text set in these fonts may extract wrongly." % f["no
_tounicode"][:5])
        if f.get("type3"):
            n.append("%d Type 3 fonts: glyph identities may be lost in extraction." % f["type3"])
        if f.get("glyphless_ocr_font") or rec.get("ocr_metadata_hint"):
```

```
            n.append("OCR indicators (GlyphLessFont or scanner/OCR producer): text layer is "
                     "probabilistic; prose is NOT automatically safe.")
        if not ch["text"]:
            n.append("PDF without a usable text layer (scanned?): image-only regime.")
        if not ch["image"]:
            n.append("Cannot render this PDF here (no pdftoppm/PyMuPDF): math unverifiable until a renderer
is available.")
    if s:
        sc = s["substitution_candidates"]
        if sc["eqtags_eth_thorn"] >= 3 or (sc["eth"] > 50 and sc["THORN"] > 50):
            n.append("SUBSTITUTION suspect: %d eth / %d THORN, %d equation tags of the form "
                     "eth-N-THORN. A remapped math font is likely; ordinary letters and digits "
                     "may also be remapped. Map every glyph on the image."
                     % (sc["eth"], sc["THORN"], sc["eqtags_eth_thorn"]))
        if sc["ket_closed_by_i"] > 20:
            n.append("SUBSTITUTION suspect: %d '|...i' patterns (closing angle bracket as 'i'?)." % sc["ket_
closed_by_i"])
        if sc["return_arrow"] > 5 or sc["dropped_ligature_words"] > 5:
            n.append("SUBSTITUTION IN PROSE suspect: ligature damage (U+21B5 or dropped fi/ff).")
        if s["short_line_fraction"] > 0.08:
            n.append("GEOMETRY suspect: %.0f%% of non-empty lines have <=2 characters "
                     "(flattened sub/superscripts)." % (100 * s["short_line_fraction"]))
        if s["control_bytes"]:
            n.append("Control bytes %s: font-slot leakage, per-document, possibly "
                     "multi-valued. Map on the image." % s["control_bytes"])
        if s["control_bytes_between_letters"]:
            n.append("Control bytes between lowercase letters %s: likely line-break "
                     "hyphens in prose (confirm one)." % s["control_bytes_between_letters"])
        if s["private_use"]:
            n.append("%d Private-Use characters: no stable meaning." % s["private_use"])
        if s["math_alphanumeric"]:
            n.append("%d mathematical-alphanumeric characters: grep them literally "
                     "(italic omega is not ASCII-searchable as w or omega)." % s["math_alphanumeric"])
        if s["line_endings"]["CRLF"]:
            n.append("CRLF line endings: strip \\r before anchoring a regex on '$'.")
        pg = rec.get("pages") or 0
        if s["form_feeds"] and cont != "pdf" and abs(s["form_feeds"] - pg) > 1:
            n.append("\\x0c occurs %d times but does not match the page count: it is a "
                     "glyph, not a page break." % s["form_feeds"])
    return n

# ---------------------------------------------------------------- page offsets


def _best_offset(name, per_page_values, n, out):
    """per_page_values: {page_index: set(printed numbers)}."""
    if not per_page_values:
        return
    score = collections.Counter()
    for i, vals in per_page_values.items():
        for v in vals:
            score[v - i] += 1
    k, agree = score.most_common(1)[0]
    out[name] = {
        "offset": k, "agreeing_pages": agree,
        "pages_with_match": len(per_page_values), "pages_total": n,
        "disagreeing": sorted((i, sorted(v)) for i, v in per_page_values.items()
                              if (i + k) not in v)[:10],
        "ambiguous_pages": sorted(i for i, v in per_page_values.items() if len(v) > 1)[:10],
        "missing_pages": [i for i in range(1, n + 1) if i not in per_page_values][:20],
    }


def folio_candidates(pages):
    n, out = len(pages), {}
    # e-locator folios, e.g. 041006-3: keep the dominant prefix only
    pref = collections.Counter(m.group(1) for t in pages
                               for m in re.finditer(r"\b(\d{6})-(\d{1,3})\b", t))
    if pref:
```

```
        top = pref.most_common(1)[0][0]
        per = {}
        for i, t in enumerate(pages, 1):
            vals = {int(m.group(1)) for m in re.finditer(r"\b%s-(\d{1,3})\b" % top, t)}
            if vals:
                per[i] = vals
        _best_offset("e-locator %s-N" % top, per, n, out)
    # "(N of T)" / "N of T": keep the dominant total
    for name, rx in (("(N of T)", r"\((\d{1,4}) of (\d{1,4})\)"),
                     ("N of T", r"(?<![\d(])(\d{1,4}) of (\d{1,4})\b")):
        tot = collections.Counter(m.group(2) for t in pages for m in re.finditer(rx, t))
        if tot:
            T = tot.most_common(1)[0][0]
            per = {}
            for i, t in enumerate(pages, 1):
                vals = {int(m.group(1)) for m in re.finditer(rx, t) if m.group(2) == T}
                if vals:
                    per[i] = vals
            _best_offset("%s, T=%s" % (name, T), per, n, out)
    # dash-framed marker on its own line: "- 184 -" or "— 61 —"
    per = {}
    for i, t in enumerate(pages, 1):
        vals = {int(m.group(1)) for m in re.finditer(
            r"(?m)^\s*[-\u2013\u2014]\s*(\d{1,4})\s*[-\u2013\u2014]\s*\r?$", t)}
        if vals:
            per[i] = vals
    _best_offset("dash-framed '- N -'", per, n, out)
    # bare number as the last non-empty line (footer folio, typical of LaTeX)
    per = {}
    for i, t in enumerate(pages, 1):
        last = next((ln.strip() for ln in reversed(t.split("\n")) if ln.strip()), "")
        if re.fullmatch(r"\d{1,4}", last):
            per[i] = {int(last)}
    _best_offset("last line is bare N", per, n, out)
    # running head: first non-empty line begins or ends with a number
    for name, rx in (("first line starts with N", r"^\s*(\d{1,4})\s+\S"),
                     ("first line ends with N", r"\S\s+(\d{1,4})\s*$")):
        per = {}
        for i, t in enumerate(pages, 1):
            first = next((ln.strip("\r") for ln in t.split("\n") if ln.strip()), "")
            m = re.search(rx, first)
            if m:
                per[i] = {int(m.group(1))}
        _best_offset(name, per, n, out)
    return out


def running_headers(text, sample=12):
    """Text-only files have no pages: look for 'N Title' / 'Title N' lines."""
    nums = []
    for ln in text.split("\n"):
        s = ln.strip("\r").strip()
        m = re.match(r"^(\d{1,4}) ((?:\d+(?:\.\d+)* )?[A-Z][A-Za-z].{3,80})$", s)
        if m:
            nums.append(int(m.group(1)))
            continue
        m = re.match(r"^((?:\d+(?:\.\d+)* )?[A-Z][A-Za-z].{3,80}) (\d{1,4})$", s)
        if m:
            nums.append(int(m.group(2)))
    if not nums:
        return {"count": 0}
    inc = sum(1 for a, b in zip(nums, nums[1:]) if 0 < b - a <= 2)
    return {"count": len(nums), "first": nums[0], "last": nums[-1],
            "monotone_fraction": round(inc / max(1, len(nums) - 1), 3),
            "sample": nums[:sample]}

# ---------------------------------------------------------------- profile
```

```
def profile_file(path):
    rec = {"name": os.path.basename(path), "path": os.path.abspath(path),
           "size": os.path.getsize(path), "sha256": sha256_of(path)}
    cont, detail = detect_container(path)
    rec.update(container=cont, container_detail=detail,
               channels={"image": False, "text": False}, pages=None)
    pages = None

    if cont == "zip":
        with zipfile.ZipFile(path) as zf:
            lay = zip_layout(zf)
            rec["zip_layout"] = lay
            if lay["paged_archive"]:
                rec["pages"] = lay["page_images"]
                rec["channels"] = {"image": True, "text": lay["page_texts"] > 0}
                if lay["manifest"]:
                    try:
                        man = json.loads(zf.read("manifest.json"))
                        dims = collections.Counter(
                            (p["image"]["dimensions"]["width"], p["image"]["dimensions"]["height"])
                            for p in man.get("pages", []))
                        rec["image_dims"] = [[w, h, k] for (w, h), k in dims.items()]
                        rec["manifest_has_visual_content_true"] = sum(
                            1 for p in man.get("pages", []) if p.get("has_visual_content"))
                    except Exception as e:  # noqa: BLE001
                        rec["manifest_error"] = str(e)
        if rec["channels"]["text"]:
            pages = zip_pages(path)
    elif cont == "pdf":
        info = {}
        if have("pdfinfo"):
            _, out, _ = run(["pdfinfo", path])
            for ln in out.splitlines():
                if ":" in ln:
                    k, v = ln.split(":", 1)
                    info[k.strip()] = v.strip()
        rec["pdfinfo"] = {k: info[k] for k in ("Pages", "Producer", "Creator",
                                               "Page size", "Encrypted") if k in info}
        rec["pages"] = pdf_npages(path)
        rec["channels"]["image"] = have("pdftoppm") or has_module("fitz")
        if have("pdffonts"):
            _, out, _ = run(["pdffonts", path])
            fonts, seen = [], set()
            for ln in out.splitlines()[2:]:
                p = ln.split()
                if len(p) >= 7 and p[0] not in seen:
                    seen.add(p[0])
                    fonts.append({"name": p[0], "type": " ".join(p[1:-5]), "uni": p[-3]})
            rec["fonts"] = {"count": len(fonts),
                            "type3": sum(1 for f in fonts if "Type 3" in f["type"]),
                            "no_tounicode": [f["name"] for f in fonts if f["uni"] == "no"][:15],
                            "glyphless_ocr_font": any("GlyphLess" in f["name"] for f in fonts),
                            "sample": [f["name"] for f in fonts][:10]}
        pc = (info.get("Producer", "") + " " + info.get("Creator", "")).lower()
        rec["ocr_metadata_hint"] = any(k in pc for k in ("ocr", "tesseract", "abbyy",
                                                         "finereader", "scan", "paper capture"))
        pages = pdf_pages(path, rec["pages"]) if rec["pages"] else None
        if pages is not None:
            per = [len(p.strip()) for p in pages]
            rec["text_chars_per_page"] = {
                "min": min(per), "median": sorted(per)[len(per) // 2],
                "sparse_pages_lt200": [i + 1 for i, v in enumerate(per) if v < 200][:30]}
            rec["channels"]["text"] = sum(per) > 100 * len(per) * 0.2
        if have("pdfimages"):
            _, out, _ = run(["pdfimages", "-list", path])
            pg = collections.Counter(int(l.split()[0]) for l in out.splitlines()[2:]
                                     if l.split() and l.split()[0].isdigit())
            rec["raster_images"] = {"pages_with_images": len(pg), "total": sum(pg.values())}
    elif cont == "transcription":
```

```
        rec["channels"]["text"] = True
        rec["transcription"] = transcription_info(path)
        pr = rec["transcription"]["pages"]
        rec["pages"] = (pr[1] - pr[0] + 1) if pr else None
    elif cont in ("text-extraction", "source-text"):
        rec["channels"]["text"] = True
        txt = get_whole_text(path)
        rec["text_signals"] = text_signals(txt)
        if cont == "text-extraction":
            rec["running_headers"] = running_headers(txt)
    elif cont == "image":
        rec["channels"]["image"] = True
        rec["pages"] = 1

    if cont == "pdf" and figures_owner(path):
        rec["figures_of"] = figures_owner(path)
        pages = None                      # labels only: no folio or text signals
    if pages is not None:
        rec["folio_candidates"] = folio_candidates(pages)
        rec["text_signals"] = text_signals("\n".join(pages))
        rec["text_chars_min_page"] = min(len(p) for p in pages) if pages else 0
    rec["notes"] = notes_for(rec)
    return rec


def print_profile(r):
    print("=" * 78)
    print(r["name"])
    print("  container : %s (%s)  size %d  sha256 %s..." % (r["container"], r["container_detail"], r["siz
e"], r["sha256"][:16]))
    print("  channels  : image=%s text=%s  pages=%s" % (r["channels"]["image"], r["channels"]["text"], r["pa
ges"]))
    if "image_dims" in r:
        print("  images    : %s  (manifest has_visual_content=true on %s pages)" % (r["image_dims"], r.get
("manifest_has_visual_content_true")))
    if "pdfinfo" in r:
        print("  pdfinfo   : %s" % r["pdfinfo"])
    if "transcription" in r:
        t = r["transcription"]
        print("  transcr.  : of %s ; pages %s (%d markers, %d with printed numbers) ; "
              "%d \\tag ; %d figure lines -> %s"
              % (t["source_pdf"], "%d-%d" % t["pages"] if t["pages"] else "none",
                 t["page_markers"], t["printed_given"], t["tags"], t["figure_lines"],
                 t["figure_files"] or "-"))
    if "fonts" in r:
        f = r["fonts"]
        print("  fonts     : %d  Type3=%d  no-ToUnicode=%d  GlyphLess=%s  e.g. %s" % (f["count"], f["type
3"], len(f["no_tounicode"]), f["glyphless_ocr_font"], f["sample"][:4]))
    s = r.get("text_signals")
    if s:
        print("  text      : %d chars, %d lines, endings %s, form feeds %d" % (s["chars"], s["lines"], s["li
ne_endings"], s["form_feeds"]))
        print("  glyphs    : ctl %s | PUA %d | math-alnum %d | combining %d | U+2212 %d | macron %d" % (s["c
ontrol_bytes"], s["private_use"], s["math_alphanumeric"], s["combining_marks"], s["unicode_minus"], s["free_
standing_macron"]))
        print("  subst     : %s" % s["substitution_candidates"])
        print("  geometry  : short-line fraction %.3f ; eq-tag styles %s" % (s["short_line_fraction"], s["eq
tag_styles"]))
    for k, v in r.get("folio_candidates", {}).items():
        print("  folio     : [%s] offset %+d agrees on %d of %d matched (of %d pages); disagree %s; missing
%s"
              % (k, v["offset"], v["agreeing_pages"], v["pages_with_match"], v["pages_total"], v["disagreein
g"][:3], v["missing_pages"][:6]))
    if r.get("running_headers", {}).get("count"):
        print("  headers   : %s" % r["running_headers"])
    for n in r["notes"]:
        print("  NOTE      : " + n)

# ------------------------------------------------------------------ prose sketch
```

```
#
# A rendition swap replaces a file by another rendering of the same document
# (e.g. a genuine PDF by a paged ZIP of page images plus text layers). Hashes
# then differ although the content does not. The sketch tests "same document"
# across text extractors: per page, a 64-bit simhash of the set of letter
# 5-grams of the lowercased, NFKC-normalised letters-only text stream.
# Letters-only shingles survive lost spaces, line-break hyphens, ligatures and
# column interleaving; math damage contributes noise, not signal.
#
# Calibration (2026-10-02, 9 papers/notes, 258 pages): median per-page Hamming
# distance between pdftotext and pypdf extractions of the same file <= 7.5;
# pdftotext vs Tesseract OCR of the page image 12-15; different documents >= 26.
# The sketch identifies the DOCUMENT, not the VERSION: a revision that edits a
# few equations or sentences is invisible to it. It is evidence for a swap,
# never proof; an unknown hash is always confirmed with the user.

SKETCH_ALGO = "simhash64-letters5"
SKETCH_K = 5
SKETCH_MIN_SHINGLES = 200      # pages with less prose (figures, blanks) -> null
SWAP_AGREE_MAX = 16            # page-median Hamming <= this: same document
SWAP_DIFFER_MIN = 22           # page-median Hamming >= this: different document
SWAP_MIN_PAGES = 3             # fewer comparable pages: inconclusive
DOC_AGREE_MAX = 10             # whole-document fallback (non-paged files)


def _shingles(text):
    s = re.sub(r"[^a-z]", "", unicodedata.normalize("NFKC", text).lower())
    return {s[i:i + SKETCH_K] for i in range(len(s) - SKETCH_K + 1)}


def _simhash(shingles):
    if len(shingles) < SKETCH_MIN_SHINGLES:
        return None
    v = [0] * 64
    for w in shingles:
        h = int.from_bytes(hashlib.blake2b(w.encode(), digest_size=8).digest(), "big")
        for b in range(64):
            v[b] += 1 if (h >> b) & 1 else -1
    return "%016x" % sum(1 << b for b in range(64) if v[b] > 0)


def prose_sketch(path):
    """Sketch of a document file, or None if it has no text channel."""
    cont, _ = detect_container(path)
    pages = get_pages(path)
    if pages is not None:
        whole = "\n".join(pages)
        per = [_simhash(_shingles(t)) for t in pages]
    elif cont in ("text-extraction", "source-text", "transcription"):
        whole, per = get_whole_text(path), None
    else:
        return None
    return {"algo": SKETCH_ALGO, "doc": _simhash(_shingles(whole)), "pages": per}


def _ham(a, b):
    return bin(int(a, 16) ^ int(b, 16)).count("1")


def compare_sketches(new, old):
    """('agree'|'differ'|'inconclusive', detail). Neutral evidence only."""
    if not new or not old or old.get("algo") != new.get("algo"):
        return "inconclusive", "no comparable reference sketch"
    if new["pages"] is not None and old.get("pages") is not None:
        if len(new["pages"]) != len(old["pages"]):
            return "differ", "page counts differ"
        hs = [_ham(a, b) for a, b in zip(new["pages"], old["pages"]) if a and b]
        if len(hs) < SWAP_MIN_PAGES:
            return "inconclusive", "only %d comparable pages" % len(hs)
```

```
        med = sorted(hs)[len(hs) // 2]
        detail = "page-median Hamming %d over %d pages" % (med, len(hs))
        if med <= SWAP_AGREE_MAX:
            return "agree", detail
        if med >= SWAP_DIFFER_MIN:
            return "differ", detail
        return "inconclusive", detail
    if new.get("doc") and old.get("doc"):
        d = _ham(new["doc"], old["doc"])
        return ("agree" if d <= DOC_AGREE_MAX else "inconclusive"), "whole-document Hamming %d" % d
    return "inconclusive", "no comparable reference sketch"

# ------------------------------------------------------------------ inventory / check


def rendition_of(path, with_sketch=True):
    cont, _ = detect_container(path)
    pages = None
    if cont == "zip":
        with zipfile.ZipFile(path) as zf:
            lay = zip_layout(zf)
        pages = lay["page_images"] if lay["paged_archive"] else None
    elif cont == "pdf":
        pages = pdf_npages(path) or None
    r = {"sha256": sha256_of(path), "size": os.path.getsize(path),
         "container": cont, "pages": pages,
         "first_seen": datetime.date.today().isoformat()}
    if cont == "transcription":
        r["source_pdf"] = detect_container(path)[1]
        pr = transcription_info(path)["pages"]
        r["pages"] = (pr[1] - pr[0] + 1) if pr else None
    owner = figures_owner(path) if cont == "pdf" else None
    if owner:
        r["role"], r["pairs_with"] = "figures", owner
    if with_sketch and cont not in SOURCE_LIKE and not owner:
        sk = prose_sketch(path)
        if sk:
            r["sketch"] = sk
    return r


def make_inventory(paths, root, old=None, add_rendition=()):
    """Inventory of PATHS. With OLD (a loaded inventory), keep known renditions.

    Per file: hash already recorded -> OLD entry kept (a missing sketch is
    filled in for the mounted rendition); new hash and name in ADD_RENDITION ->
    appended as a further rendition; new hash otherwise -> the entry is
    replaced (old renditions retired: the content is treated as new).
    """
    oldrec = {f["name"]: f for f in (old or {}).get("files", [])}
    files, log = [], []
    for p in paths:
        n = os.path.basename(p)
        new = rendition_of(p)
        prev = oldrec.get(n)
        if prev is None:
            files.append({"name": n, "renditions": [new]})
            log.append("new       : " + n)
            continue
        rends = [dict(r) for r in prev["renditions"]]
        hit = next((r for r in rends if r["sha256"] == new["sha256"]), None)
        if hit is not None:
            if "sketch" not in hit and "sketch" in new:
                hit["sketch"] = new["sketch"]
            files.append({"name": n, "renditions": rends})
            log.append("kept      : %s (%s rendition known)" % (n, hit["container"]))
        elif n in add_rendition:
            files.append({"name": n, "renditions": rends + [new]})
            log.append("rendition+: %s (%s added beside %s)"
```

```
                      % (n, new["container"], ", ".join(r["container"] for r in rends)))
        else:
            files.append({"name": n, "renditions": [new]})
            log.append("replaced  : %s (old renditions retired)" % n)
    for n in sorted(set(oldrec) - {os.path.basename(p) for p in paths}):
        log.append("dropped   : " + n)
    for ln in log:
        print(ln, file=sys.stderr)
    return {"pdf_protocol_inventory": {"triage_version": TRIAGE_VERSION, "format": 2,
                                       "generated": datetime.date.today().isoformat(),
                                       "root": root, "files": files}}


def _normalise_entry(f):
    """Format-1 entries (one hash per file) become a single rendition."""
    if "renditions" in f:
        return f
    r = {k: f.get(k) for k in ("sha256", "size", "container", "pages")}
    return {"name": f["name"], "renditions": [r]}


def load_inventory(path):
    with open(path, encoding="utf-8") as f:
        txt = f.read()
    obj = None
    if path.lower().endswith(".json"):
        obj = json.loads(txt)
    else:
        for m in re.finditer(r"```json\s*\n(.*?)\n```", txt, re.S):
            if "pdf_protocol_inventory" in m.group(1):
                obj = json.loads(m.group(1))
                break
        if obj is None:
            raise ValueError("no ```json block containing 'pdf_protocol_inventory'")
    inv = obj.get("pdf_protocol_inventory", obj)
    inv["files"] = [_normalise_entry(f) for f in inv["files"]]
    return inv


def classify_unknown(path, rends):
    """An unrecorded hash under a known name: 'swap' or 'changed', with reason."""
    cont, _ = detect_container(path)
    cur = rendition_of(path)
    known = {r["container"] for r in rends}
    if cont in known:
        sk = [compare_sketches(cur.get("sketch"), r.get("sketch"))
              for r in rends if r["container"] == cont]
        extra = "; sketch: %s (%s)" % sk[0] if sk and sk[0][0] == "agree" else ""
        return "changed", "same container (%s), new hash%s" % (cont, extra)
    if cont in SOURCE_LIKE or known & set(SOURCE_LIKE):
        return "changed", "container %s -> %s" % ("/".join(sorted(known)), cont)
    paged = [r for r in rends if r.get("pages")]
    if cur["pages"] and paged:
        if all(r["pages"] != cur["pages"] for r in paged):
            return "changed", "container %s -> %s, pages %s -> %d" % (
                "/".join(sorted(known)), cont,
                "/".join(str(r["pages"]) for r in paged), cur["pages"])
        refs = [r for r in paged if r["pages"] == cur["pages"]]
    elif not cur["pages"] and not paged:
        refs = list(rends)
    else:
        return "changed", "container %s -> %s, paged vs non-paged" % (
            "/".join(sorted(known)), cont)
    base = "container %s -> %s, %s" % ("/".join(sorted(known)), cont,
                                       "%d pages match" % cur["pages"] if cur["pages"] else "both non-page
d")
    sks = [r for r in refs if r.get("sketch")]
    if not sks:
        return "swap", base + "; no reference sketch (structure only)"
```

```
    verdicts = [compare_sketches(cur.get("sketch"), r["sketch"]) for r in sks]
    if any(v[0] == "agree" for v in verdicts):
        return "swap", base + "; sketch: agree (%s)" % next(v[1] for v in verdicts if v[0] == "agree")
    return "changed", base + "; sketch: %s (%s)" % verdicts[0]


def check(inv_path, root=None, as_json=False):
    """Compare the inventory with the files on disk.

    A file is unchanged if its hash matches ANY recorded rendition. An
    unrecorded hash is a probable rendition swap when the container differs
    from every recorded rendition, the page count matches, and the prose
    sketch (if one is recorded) agrees; otherwise it is changed. Swaps alone
    do not make the protocol stale (exit 3), but they always need the user's
    confirmation, and the protocol holds no reading facts for the new
    rendition until it is updated.

    Only document files (PDF, ZIP, text extraction, image, unknown) can make a
    protocol stale. Source-like files need no reading facts -- they are read
    directly -- so adding, editing or removing one is reported separately and
    does not count as stale. Source-like means: source text (.tex .md .bib
    .sty .cls), a pdf-to-markdown transcription (recognised by its legend,
    whatever its extension), and the figures PDF lying next to a
    transcription (NAME_figures.pdf beside NAME.md). A file that changes
    between a source-like kind and a document kind does count.

    A recorded document that is gone while a transcription declaring it as
    its source is present (names compared by norm_stem) is reported CONVERTED.
    It is stale like any removal: the protocol entry still carries reading
    facts for a rendition that no longer exists. A new transcription whose
    source is still recorded and mounted is flagged as pairing with it.
    """
    inv = load_inventory(inv_path)
    root = root or inv.get("root") or "."
    rec = {f["name"]: f["renditions"] for f in inv["files"]}
    srcish = lambda r: r.get("container") in SOURCE_LIKE or r.get("role") == "figures"  # noqa: E731
    is_src = lambda rs: all(srcish(r) for r in rs)  # noqa: E731
    disk = {n for n in os.listdir(root) if os.path.isfile(os.path.join(root, n))}
    ignored = {n for n in disk if re.fullmatch(r"pdf_protocol_.*\.md", n)}
    kind = {n: file_kind(os.path.join(root, n)) for n in disk - ignored}
    src_kinds = ("source-text", "transcription", "figures")
    label = {"source-text": "source", "transcription": "transcr.", "figures": "figures"}
    # transcriptions on disk, by the normalised name of the PDF they transcribe
    tr_of = {}
    for n, k in kind.items():
        if k == "transcription":
            tr_of.setdefault(norm_stem(detect_container(os.path.join(root, n))[1]), n)
    added, added_src, removed, removed_src = [], [], [], []
    changed, changed_src, swaps, same, mounted = [], [], [], [], []
    converted, pairs = [], []
    for n in sorted(disk - set(rec) - ignored):
        if kind[n] in src_kinds:
            added_src.append({"name": n, "kind": kind[n]})
            if kind[n] == "transcription":
                tgt = [m for m in rec if m in disk and not is_src(rec[m])
                       and norm_stem(m) == norm_stem(detect_container(os.path.join(root, n))[1])]
                if tgt:
                    pairs.append({"transcription": n, "document": tgt[0]})
        else:
            added.append(n)
    for n in sorted(set(rec) - disk):
        if is_src(rec[n]):
            removed_src.append(n)
        elif norm_stem(n) in tr_of:
            converted.append({"name": n, "transcription": tr_of[norm_stem(n)]})
        else:
            removed.append(n)
    for n in sorted(set(rec) & disk):
        p = os.path.join(root, n)
```

```
        size, digest, rs = os.path.getsize(p), sha256_of(p), rec[n]
        hit = next((r for r in rs if r.get("size") == size and r.get("sha256") == digest), None)
        if hit is not None:
            same.append(n)
            if len(rs) > 1:
                mounted.append((n, hit["container"], len(rs)))
            continue
        new = kind[n]
        if is_src(rs) and new in src_kinds:
            changed_src.append({"name": n, "kind": new})
            continue
        verdict, why = classify_unknown(p, rs)
        item = {"name": n, "known": sorted({r["container"] for r in rs}), "new_container": new,
                "reason": why}
        (swaps if verdict == "swap" else changed).append(item)
    stale = bool(added or removed or changed or converted)
    res = {"root": root, "inventory_generated": inv.get("generated"), "unchanged": same,
           "mounted_renditions": [{"name": n, "container": c, "known": k} for n, c, k in mounted],
           "added": added, "removed": removed, "converted": converted, "changed": changed,
           "probable_swaps": swaps,
           "sources_added": [a["name"] for a in added_src],
           "sources_removed": removed_src,
           "sources_changed": [c["name"] for c in changed_src],
           "source_kinds": {a["name"]: a["kind"] for a in added_src + changed_src},
           "transcription_pairs": pairs, "stale": stale}
    if as_json:
        print(json.dumps(res, indent=1, ensure_ascii=False))
    else:
        print("staleness check against inventory of %s (root %s)" % (inv.get("generated"), root))
        print("  unchanged : %d file(s)" % len(same))
        by = collections.Counter(c for _, c, _ in mounted)
        for c, k in sorted(by.items()):
            print("  mounted   : %s rendition, for %d file(s) with several recorded "
                  "(use that rendition's facts)" % (c, k))
        for n in added:
            print("  ADDED     : " + n)
        for n in removed:
            print("  REMOVED   : " + n)
        for c in converted:
            print("  CONVERTED : %s -> %s (original gone; replace its entry by a transcription "
                  "entry, references/project_protocol.md 6)" % (c["name"], c["transcription"]))
        for c in changed:
            print("  CHANGED   : %s (%s)" % (c["name"], c["reason"]))
        for c in swaps:
            print("  SWAP?     : %s (%s)" % (c["name"], c["reason"]))
        how = {"figures": "view pages on demand; not stale"}
        for a in added_src:
            print("  %-8s+ : %s (%s)" % (label[a["kind"]], a["name"],
                                          how.get(a["kind"], "read directly; not stale")))
        for q in pairs:
            print("  pairing   : %s transcribes %s, which is still mounted (read the "
                  "transcription for content; mention it to the user)"
                  % (q["transcription"], q["document"]))
        for n in removed_src:
            print("  source -  : %s (not stale; update any pairing that names it)" % n)
        for c in changed_src:
            print("  %-8s~ : %s (edited; read directly; not stale)" % (label[c["kind"]], c["name"]))
        if stale:
            result = "STALE -- re-triage the document files listed above"
            if converted and not (added or removed or changed):
                result = "STALE -- only conversions: rewrite those entries (no triage needed)"
        elif swaps:
            result = ("PROBABLE RENDITION SWAP -- content likely unchanged, but the protocol "
                      "has no reading facts for these renditions; read them via the "
                      "no-protocol path and confirm with the user before recording them")
        else:
            result = "up to date" + (" (source-file changes listed above)"
                                     if added_src or removed_src or changed_src else "")
        print("  RESULT    : " + result)
```

```
    return 1 if stale else (3 if swaps else 0)

# ---------------------------------------------------------------- text / grep


def cmd_text(path, page, lines, esc):
    if detect_container(path)[0] == "transcription" and page:
        segs = [t for n, _, t in transcription_pages(get_whole_text(path).replace("\r", ""))
                if n == page]
        if not segs:
            sys.exit("no page marker for page %d in this transcription" % page)
        print("".join(segs).strip("\n"))
        return
    pages = get_pages(path)
    if pages is not None:
        if not page:
            sys.exit("paged file: give a page number")
        if not 1 <= page <= len(pages):
            sys.exit("page out of range 1..%d" % len(pages))
        t = pages[page - 1]
    else:
        t = get_whole_text(path)
        if lines:
            a, b = (int(x) if x else None for x in lines.split(":"))
            t = "\n".join(t.split("\n")[(a - 1 if a else 0):b])
    print(escape(t) if esc else t.replace("\r", ""))


def cmd_grep(pattern, files, icase, esc):
    rx = re.compile(pattern, re.I if icase else 0)
    hits = 0
    for f in files:
        if detect_container(f)[0] == "transcription":
            hits += _grep_transcription(rx, f, esc)
            continue
        pages = get_pages(f)
        units = list(enumerate(pages, 1)) if pages is not None else [(None, get_whole_text(f))]
        for pg, t in units:
            for ln_no, ln in enumerate(t.split("\n"), 1):
                ln = ln.rstrip("\r")
                if rx.search(ln):
                    hits += 1
                    loc = ("p%d:%d" % (pg, ln_no)) if pg else ("L%d" % ln_no)
                    print("%s:%s: %s" % (os.path.basename(f)[:40], loc, escape(ln) if esc else ln))
    print("-- %d hit(s)" % hits, file=sys.stderr)



def _grep_transcription(rx, f, esc):
    """Hits in a transcription, located by its page markers: the page is the
    one in force at the start of the match (a marker can stand mid-line)."""
    hits, cur = 0, None
    for ln_no, ln in enumerate(get_whole_text(f).replace("\r", "").split("\n"), 1):
        marks = [(m.start(), int(m.group(1)), m.group(2)) for m in PAGE_MARK_RX.finditer(ln)]
        for m in rx.finditer(ln):
            pg = cur
            for pos, n, pr in marks:
                if pos <= m.start():
                    pg = (n, pr)
            hits += 1
            loc = ("p%d%s" % (pg[0], "(printed %s)" % pg[1] if pg[1] else "")) if pg else "p?"
            print("%s:%s:L%d: %s" % (os.path.basename(f)[:40], loc, ln_no, escape(ln) if esc else ln))
            break                                   # one report per line
        if marks:
            cur = (marks[-1][1], marks[-1][2])
    return hits

# ---------------------------------------------------------------- extract / crop
```

```
def cmd_extract(path, out):
    os.makedirs(out, exist_ok=True)
    cont, _ = detect_container(path)
    if cont == "zip":
        with zipfile.ZipFile(path) as zf:
            zf.extractall(out)
        print("extracted ZIP to " + out)
    elif cont == "pdf":
        if not have("pdftotext"):
            sys.exit("pdftotext unavailable")
        n = pdf_npages(path)
        for i in range(1, n + 1):
            run(["pdftotext", "-enc", "UTF-8", "-f", str(i), "-l", str(i), path,
                 os.path.join(out, "%d.txt" % i)])
        print("wrote %d per-page .txt files to %s (render pages with 'crop')" % (n, out))
    else:
        sys.exit("extract: nothing to extract for container '%s'" % cont)


def load_page_image(src, page, dpi):
    from PIL import Image  # noqa: WPS433
    cont, _ = detect_container(src)
    if cont == "image":
        return Image.open(src)
    if cont == "zip":
        with zipfile.ZipFile(src) as zf:
            for ext in ("jpeg", "jpg", "png"):
                nm = "%d.%s" % (page, ext)
                if nm in zf.namelist():
                    im = Image.open(io.BytesIO(zf.read(nm)))
                    im.load()
                    return im
        sys.exit("page %d image not in archive" % page)
    if cont == "pdf":
        if have("pdftoppm"):
            with tempfile.TemporaryDirectory() as td:
                run(["pdftoppm", "-r", str(dpi), "-f", str(page), "-l", str(page),
                     "-png", src, os.path.join(td, "pg")])
                fs = [x for x in os.listdir(td) if x.endswith(".png")]
                if not fs:
                    sys.exit("render failed")
                im = Image.open(os.path.join(td, fs[0]))
                im.load()
                return im
        if has_module("fitz"):
            import fitz  # noqa: WPS433
            pix = fitz.open(src)[page - 1].get_pixmap(dpi=dpi)
            return Image.open(io.BytesIO(pix.tobytes("png")))
        sys.exit("no PDF renderer available (pdftoppm or PyMuPDF)")
    sys.exit("crop: unsupported container '%s'" % cont)


def cmd_crop(src, page, box, scale, dpi, out):
    from PIL import Image  # noqa: WPS433
    im = load_page_image(src, page, dpi)
    W, H = im.size
    if box:
        x0, y0, x1, y1 = box
        im = im.crop((max(0, x0), max(0, y0), min(W, x1), min(H, y1)))
    if scale and scale != 1:
        im = im.resize((int(im.size[0] * scale), int(im.size[1] * scale)), Image.LANCZOS)
    im.save(out)
    print("page image %dx%d -> wrote %s (%dx%d)" % (W, H, out, im.size[0], im.size[1]))

# ---------------------------------------------------------------- main


def main(argv=None):
```

```
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest="cmd", required=True)
    sub.add_parser("tools")
    p = sub.add_parser("profile"); p.add_argument("files", nargs="+"); p.add_argument("--json")
    p = sub.add_parser("inventory"); p.add_argument("files", nargs="+")
    p.add_argument("--root"); p.add_argument("--out"); p.add_argument("--merge")
    p.add_argument("--add-rendition", nargs="+", default=[], metavar="NAME")
    p = sub.add_parser("check"); p.add_argument("inventory"); p.add_argument("--root")
    p.add_argument("--json", action="store_true")
    p = sub.add_parser("text"); p.add_argument("file"); p.add_argument("page", nargs="?", type=int)
    p.add_argument("--lines"); p.add_argument("--escape", action="store_true")
    p = sub.add_parser("grep"); p.add_argument("pattern"); p.add_argument("files", nargs="+")
    p.add_argument("-i", action="store_true"); p.add_argument("--escape", action="store_true")
    p = sub.add_parser("extract"); p.add_argument("file"); p.add_argument("--out", required=True)
    p = sub.add_parser("crop"); p.add_argument("src"); p.add_argument("--page", type=int, default=1)
    p.add_argument("--box", type=int, nargs=4); p.add_argument("--scale", type=float, default=3.0)
    p.add_argument("--dpi", type=int, default=200); p.add_argument("--out", required=True)
    a = ap.parse_args(argv)

    if a.cmd == "tools":
        print(json.dumps(tool_report(), indent=1))
    elif a.cmd == "profile":
        recs = []
        for f in a.files:
            if os.path.isfile(f):
                r = profile_file(f)
                recs.append(r)
                print_profile(r)
            else:
                print("skip (not a file): " + f, file=sys.stderr)
        if a.json:
            with open(a.json, "w", encoding="utf-8") as fh:
                json.dump({"triage_version": TRIAGE_VERSION,
                           "generated": datetime.date.today().isoformat(),
                           "tools": tool_report(), "files": recs}, fh, indent=1, ensure_ascii=False)
            print("\nfull profile written to " + a.json)
    elif a.cmd == "inventory":
        files = [f for f in a.files if os.path.isfile(f)]
        if not files:
            sys.exit("no files")
        old = load_inventory(a.merge) if a.merge else None
        unknown = set(a.add_rendition) - {os.path.basename(f) for f in files}
        if unknown:
            print("--add-rendition names files not given: %s" % sorted(unknown), file=sys.stderr)
            return 2
        inv = make_inventory(files, a.root or os.path.dirname(os.path.abspath(files[0])),
                             old, set(a.add_rendition))
        s = json.dumps(inv, indent=1, ensure_ascii=False)
        if a.out:
            with open(a.out, "w", encoding="utf-8") as fh:
                fh.write(s + "\n")
        print(s)
    elif a.cmd == "check":
        try:
            return check(a.inventory, a.root, a.json)
        except Exception as e:  # noqa: BLE001
            print("check failed: %s" % e, file=sys.stderr)
            return 2
    elif a.cmd == "text":
        cmd_text(a.file, a.page, a.lines, a.escape)
    elif a.cmd == "grep":
        cmd_grep(a.pattern, a.files, a.i, a.escape)
    elif a.cmd == "extract":
        cmd_extract(a.file, a.out)
    elif a.cmd == "crop":
        cmd_crop(a.src, a.page, a.box, a.scale, a.dpi, a.out)
    return 0
```

```
if __name__ == "__main__":
    sys.exit(main())
```